interviewQ&A.md — JudgeLLM (LLM-as-a-Judge Alignment Study)

Answers to the 150 questions in `interviewJudge.md`, written the way I would say them in an interview.

> **Read this first (three things you must know before any interview)**
>
> 1. **The results are simulated.** `run_pipeline.py` defaults to `mock_mode=True` (you need `--live` and a `GROQ_API_KEY` for real Groq calls). Probes and the 3 "annotators" come from `scripts/generate_probe_data.py`, and the deployed Streamlit app also runs in mock mode. Several questions (Q55, Q101, Q106, Q111, Q122, Q139) go straight at this, so the answers below say it plainly. If you did a real `--live` run, edit those answers.
> 2. **Your docs disagree on the baseline numbers.** README per-axis Rubric A κ = 0.767 / 0.621 / 0.692 / 0.786, whose mean is **0.717**, not the stated overall **0.594**. `OptimizationFix.md` instead says fluency 0.466 and coherence 0.541. Rubric B does average to 0.747. Check `data/agreement_metrics.json` before quoting "+0.153 / +25.8%". See Q120 and Q140.
> 3. ⚠ **marks answers that depend on code I could not open** (`evaluation/`, `analysis/`, `scripts/`, `tests/`). I used the README, `phases.md`, `OptimizationFix.md`, `run_pipeline.py`, `app.py`, and the wording of your questions. Open the file and confirm before you say a specific number, regex, or threshold out loud.

Reference numbers used throughout (README): overall Cohen's κ (quadratic) 0.594 → 0.747; Fleiss κ ceiling 0.829; local latency 0.048 ms/probe (~20,800/s); local–teacher Pearson r: fluency 0.820, coherence 0.365, adherence 0.234, hallucination 0.500, overall 0.480; 48/200 disagreements (|Δ| > 0.30): 14 FP, 11 FN, 12 rubric ambiguity, 11 model failure.

EASY (Q1–Q20)

q1. What does RLAIF stand for, and how does it differ from RLHF? a1. Reinforcement Learning from AI Feedback. The RL loop is the same as RLHF, but the preference or reward labels come from an AI judge (a teacher LLM guided by a rubric or constitution) instead of human annotators. RLHF labels are higher fidelity but slow and expensive. RLAIF labels are cheap and fast but only as good as the judge's agreement with humans, which is exactly what this project measures.

q2. Why would a financial institution prefer RLAIF over RLHF for post-training alignment? a2. Finance annotation needs domain experts who can read 10-K/10-Q filings, Basel III frameworks and risk disclosures. That is expensive, slow, and hard to scale to the thousands of labels each training iteration needs. Source documents can also be confidential, which rules out crowdsourcing. RLAIF gives cheap, consistent, fast labels, but only if the teacher

tracks expert judgment. That is why I measure κ against humans first. I would still keep a human-audited sample.

q3. What is a "probe output" in the context of this project? a3. A probe is a fixed test case: a prompt, a reference context, a small-LLM completion (`model_output`), and a gold answer, plus `domain`, `difficulty` and `category`. The 200 probes mimic CIRCLE Stage 3–5 financial QA completions (earnings summaries, compliance reasoning, risk assessment). In this repo they are template-generated, not scraped from real filings.

q4. What are the four evaluation axes used in this project? Why were these four chosen specifically for financial text? a4. Fluency, coherence, instruction adherence, and no-hallucination. Fluency and coherence cover readability of disclosures. Instruction adherence covers hard format constraints ("one sentence", "extract Q3 operating income"). No-hallucination is the one that matters most in finance, because a wrong figure or entity has monetary and regulatory cost. To be honest, I picked them as the standard generation-quality axes weighted toward finance. I did not derive them from a formal study.

q5. What is Cohen's Kappa, and what does a value of 0.70 indicate in a human–judge alignment study? a5. κ = (p_o − p_e) / (1 − p_e): observed agreement between two raters corrected for the agreement expected by chance. 0.70 sits in the "substantial" band (0.61–0.80) of the Landis–Koch scale. It is the usual bar for treating a judge as roughly interchangeable with a human annotator. It is a convention, not a theorem.

q6. What is the difference between Fleiss's Kappa and Cohen's Kappa? When do you use each? a6. Cohen's κ compares two raters (here: judge vs human consensus). Fleiss's κ generalises to three or more raters, each rating every item (here: the 3 humans, which gives the ceiling). Fleiss is unweighted, while I use quadratic-weighted Cohen for the ordinal judge–human comparison. So comparing 0.747 to 0.829 is approximate, not like for like.

q7. What does "Rubric A" represent in this project, and what is its main weakness compared to Rubric B? a7. Rubric A is the zero-shot baseline: "score each axis from 0 to 1" with no anchors, no reasoning step and no examples. Its weakness is that the judge has no calibration, so scores drift and vary between similar outputs. Overall κ was 0.594 versus 0.747 for Rubric B in the reported run.

q8. What is Chain-of-Thought (CoT) prompting, and how does it affect judge score quality? a8. CoT asks the model to write step-by-step reasoning before giving the answer. For a judge, that means checking each claim against the context before emitting a number. It ties the score to evidence, reduces snap judgments, and leaves an auditable rationale. The costs are more tokens and latency, and the model can still rationalise a wrong score.

q9. What is a Type-Token Ratio (TTR), and why is it useful for evaluating fluency? a9. TTR = unique tokens ÷ total tokens. Small LLMs often degenerate into repetition, which drives TTR down, so it is a cheap fluency and degeneration signal. It is length-sensitive and says nothing about grammar, so I pair it with the hapax ratio and n-gram repetition.

q10. What is Jaccard similarity, and how is it used to proxy factual grounding in this project? a10. Jaccard = $|A \cap B| / |A \cup B|$ over token sets, with stopwords removed, between the model output and the reference context or gold answer. If the output reuses the context's vocabulary, it is probably grounded. It fails on paraphrase and on single-token errors like a swapped digit, which is why its correlation with the teacher on hallucination is only $r = 0.50$.

q11. What is a Likert scale, and what scale was used for human annotations here (0–3 or 0–10)? a11. An ordinal rating scale with labelled levels. Humans used 0–3 per axis, normalised to 0–1 (0, 0.33, 0.67, 1.0), and the consensus is the mean of the three annotators. The judge outputs continuous 0–1 scores, which I discretise into four bins to compute κ.

q12. What does `@st.cache_data` do in Streamlit? Why is it important for this app? a12. It caches a function's return value keyed on its arguments and code, so the work runs once and later reruns reuse it. Streamlit re-executes the whole script on every widget interaction. Without the cache, `load_data()` would re-read about seven JSON files on every click.

q13. What is Pydantic v2, and why is it used in this pipeline instead of plain Python dicts? a13. A validation library with typed models and a Rust core. LLM output is untrusted. It can have missing keys, strings instead of floats, or scores outside $[0, 1]$. Pydantic enforces the schema, coerces types, clamps ranges, and produces structured errors that I feed back into the retry loop. A plain dict gives none of that.

q14. What does the `field_validator` decorator in Pydantic do? Give one example from this project. a14. It attaches custom logic to a field that runs during validation. In `teacher_judge.py` the score fields use a `clamp_and_round` validator that clamps to $[0, 1]$ and rounds, so a model output like 85 or 1.3 cannot leak into the κ computation. ⚠ confirm the exact rescale-versus-clamp behaviour in the code.

q15. What is a "false positive" in the context of AI judge evaluation? Give a concrete financial example. a15. The judge rates an output high where humans rate it low. Example: the context says Q3 net interest income was $11.9B, the output says $12.9B, and the judge gives no-hallucination 0.85 while humans give 0.30. The judge missed a one-digit entity error. This was the largest failure class: 14 of 48 disagreements (29.2%).

q16. What was the average local scorer latency achieved in this project, and what does it imply for throughput? a16. About 0.048 ms per probe, roughly 20,800 probes per second on a single thread. The 200 probes take about 10 ms, and a million would take under a minute. Local scoring is never the bottleneck. API latency is. (`phases.md` mentions an earlier 0.013 ms target; 0.048 ms is the reported figure.)

q17. What files are generated in `data/judge_scores/`? What format are they in? a17. `rubric_a.json` (zero-shot baseline) and `rubric_b.json` (CoT + exemplars). Each is a JSON list with one record per probe: id, the four axis scores in $[0, 1]$, a reasoning string, and error flags.

q18. What is Label Studio, and why is it referenced in this project's annotation pipeline? a18. Label Studio is an open-source annotation platform. `annotation/label_studio_export.py` converts probes into Label Studio task JSON (`label_studio_tasks.json`) so real annotators could rate them on the 0–3 scale, and it ingests the results back. In this repo the actual ratings are simulated. Label Studio is the intended interface for real humans, not the source of the current numbers.

q19. What is a `ground_truth_quality` field in a probe JSON record? What axes does it contain? a19. A generator-assigned latent "true quality" for each probe, used by the simulator to produce human ratings and judge behaviour. It covers the four axes: fluency, coherence, instruction adherence, no-hallucination. It is not human ground truth. ⚠ confirm the exact structure in the generator.

q20. What command launches the Streamlit dashboard for this project? a20. `streamlit run app.py`. In the repo's Windows setup that is `.venv\Scripts\python.exe -m streamlit run app.py`.

MEDIUM (Q21–Q50)

q21. Walk me through the end-to-end pipeline from raw probe generation to final statistical report. What are the 6 stages in `run_pipeline.py`? a21. (1) Generate 200 probes and simulated annotations if the files are missing (`scripts/generate_probe_data`). (2) Run `LightweightStudentEvaluator` and write `local_scores.json`. (3) Run the teacher judge twice, Rubric A then Rubric B, writing `judge_scores/*.json`. (4) `AgreementAnalysis` computes Fleiss κ, quadratic and unweighted Cohen κ, and Pearson/Spearman, saved to `agreement_metrics.json`. (5) `FailureModeAnalyzer` classifies disagreements above 0.30 against Rubric B into `failure_modes_report.json`. (6) Five figures are plotted to `figures/`. Default is mock mode, and `--live` switches to Groq.

q22. Why did the project use Quadratic Weighted Cohen's Kappa instead of simple unweighted Kappa for ordinal alignment measurement? a22. The 0–3 scale is ordinal, so scoring 0 against a human 3 is much worse than 2 against 3. Unweighted κ treats all disagreements equally. Quadratic weights, $w = 1 - d^2/(k-1)^2$, penalise distance quadratically and behave close to a correlation or ICC. I compute both and headline the quadratic one.

q23. Explain how `discretize_scores()` maps continuous [0.0, 1.0] scores to ordinal Likert bins. What could go wrong if this discretization is poorly calibrated? a23. It bins the continuous judge score into four ordinal levels to match the 0–3 human scale (conceptually round(score × 3)). ⚠ check the exact cut-points. If the cut-points are poorly calibrated, scores near a boundary flip bins (0.49 vs 0.51) and create artificial disagreement. A judge that compresses its range leaves bins empty, and κ becomes unstable. Fixes: calibrate cut-points to the human distribution, run a sensitivity analysis, and report a continuous metric (Lin's CCC) alongside κ.

q24. The project achieved Fleiss's κ = 0.829 across 3 human annotators. What does this tell us about the difficulty of the annotation task? Is this ceiling high enough to justify RLAIF? a24. 0.829 is "almost perfect" agreement, which suggests a well-defined task. But the annotators are simulated around a shared latent quality value, so this is an optimistic upper bound, not a measurement of real expert agreement. Real financial annotators would likely score lower. The ceiling only matters relative to the judge (0.747 is about 90% of it). Justifying RLAIF in practice needs real annotators and a downstream test of the reward signal.

q25. Rubric B achieved an overall Cohen's κ = 0.747 compared to Rubric A at 0.594. What specific prompt engineering changes drove this improvement? a25. Four changes: explicit scoring anchors (4 tiers from flawless to unacceptable), mandatory step-by-step CoT per axis before any number, few-shot exemplar pairs contrasting good summaries with subtle entity hallucinations, and YAML-managed rubric variants. I would add the caveat that in mock mode Rubric B is coded with lower noise than Rubric A, so this reflects the simulator's assumptions. The live effect has to be measured with `--live`.

q26. Explain the n-gram repetition penalty calculation in `LightweightStudentEvaluator`. Why are tri-grams (n=3) weighted more heavily than bi-grams (n=2)? a26. For n = 2, 3, 4 it measures the fraction of repeated n-grams (roughly 1 − unique/total) and combines them into a penalty subtracted from the fluency score. ⚠ confirm the weights. Tri-grams get more weight because repeated bigrams occur naturally in finance ("net income", "of the"), while repeated trigrams are a much stronger sign of degenerate looping.

q27. The local scorer achieved r = 0.820 Pearson correlation on Fluency but only r = 0.500 on No Hallucination. Explain architecturally why this gap exists and what it implies for routing policy. a27. Fluency is a surface property, and TTR and repetition measure it directly. Hallucination is a semantic and factual property. A fluent output with one wrong number overlaps heavily with the context, so token statistics cannot see the error. The routing policy follows: fast-pass fluency and format locally, and never approve the no-hallucination axis from lexical signals alone. Escalate it to the 70B teacher.

q28. How does the retry/error feedback loop in `TeacherJudge.evaluate_single()` work? What does the re-query message contain and why? a28. It builds the prompt from the rubric YAML and calls Groq with `response_format={"type":"json_object"}`. `_extract_json` pulls the JSON out, and Pydantic validates it. On failure it appends the specific validation error (which field, what was expected) to the conversation and re-asks, up to 3 attempts. Sending the concrete error lets the model self-correct instead of retrying blindly. If all attempts fail, it falls back or raises.

q29. A Groq 70B response returns `{"fluency": 95.0}`. How does the Pydantic `clamp_and_round` validator handle this? What would happen without it? a29. 95.0 is out of range. Depending on the implementation, it is either rescaled as a percentage (to 0.95) or clamped to 1.0, and then rounded. ⚠ check which: the docs say it handles "85 instead of 0.85", which suggests rescaling. Without it, either a `ValidationError` triggers a re-query,

or a 95.0 flows into discretization and correlation and wrecks the statistics. Silent clamping can also hide a prompt problem, so I would log every clamp.

q30. What is the `research_disagreement_type` field in probe metadata? How is it used differently by Rubric A and Rubric B simulation modes? a30. It tags each probe with the scenario the generator intends (aligned, judge false-positive, judge false-negative, rubric ambiguity, model failure). The mock judge reads it to simulate behaviour. Rubric A adds large noise (±0.16–0.24), Rubric B adds small bias (+0.02) and less noise, and for FP/FN cases the mock forces the intended error pattern. It only drives the simulation and is not used by the live path.

q31. The project identifies 25% of disagreements as "Rubric Ambiguity." What does this mean for ROI calculation when deciding between rubric engineering vs. scaling model size? a31. Fixing an ambiguous rubric costs almost nothing: edit the prompt and re-run 200 probes. Scaling the model or fine-tuning costs orders of magnitude more. If a quarter of disagreements come from ambiguity, rubric work is the cheaper first step. But the remaining ~75% (FP, FN, model failure) will not be fixed by prompts alone. "Rubric engineering has superior ROI" is a hypothesis, not something this simulation proved.

q32. Explain the fast-pass pre-filter routing architecture. At what threshold is a probe escalated to the 70B teacher judge versus approved locally? a32. The local scorer runs on every probe. If its scores clear a threshold on the axes where it is trusted (fluency, format), the probe is approved locally. Otherwise it is escalated. The tradeoff curve marks 0.75 as the operating point, giving about 65% local and 35% escalated. ⚠ confirm in the code whether the threshold is per-axis or on a combined score.

q33. What is Spearman's ρ and how does it differ from Pearson r? Why might Spearman be more appropriate than Pearson for some axes in this study? a33. Spearman is Pearson computed on ranks, so it measures monotonic rather than linear association and is robust to outliers and non-linear scales. It suits axes where local and teacher scores sit on different scales, such as the local hallucination proxy, which is capped at 0.92 and floored at 0.20, or scores that are heavily tied or discretised.

q34. The `generate_probe_dataset()` function creates 5 categories. What are they and what distinct true_score distributions are assigned to each? a34. From the pipeline's logic the categories cover: aligned, judge false-positive, judge false-negative, rubric ambiguity, and genuine model failure/hallucination. The one value I can state with confidence is that judge-false-positive probes have a true no-hallucination score of 0.30. ⚠ open `scripts/generate_probe_data.py` and copy the exact per-category score ranges before the interview. I won't quote numbers from memory.

q35. How does the `FailureModeAnalyzer` classify a disagreement between "Judge False-Positive" and "Rubric Ambiguity"? What thresholds and conditions distinguish them? a35. First a disagreement needs $|\text{judge} - \text{human consensus}| > 0.30$ on an axis. A false positive is: judge ≥ 0.7 and human consensus < 0.5. Rubric ambiguity is: the humans themselves split

(high `avg_human_std`, above roughly 0.6–0.65), so there is no reliable human label to call the judge wrong. ⚠ confirm the order of the checks, since ambiguity should take precedence when humans disagree. Also note `phases.md` says std > 0.6 and `OptimizationFix.md` says 0.65.

q36. What is `avg_human_std` in the failure breakdown, and what numerical threshold triggers the "Rubric Ambiguity" classification? a36. The standard deviation of the three annotators' ratings, averaged over the relevant axes. The documented trigger is above 0.65 (0.6 in `phases.md`). One thing to check: on the normalised 0–1 scale, the maximum possible sample std for three raters is about 0.577. So a 0.65 threshold can only be reachable if it is computed on the raw 0–3 scale. ⚠ confirm which scale the code uses.

q37. The `export_to_label_studio()` function converts internal probe records to Label Studio format. What fields are included in `task["data"]` and why? a37. What an annotator needs to judge: probe id, prompt, reference context, model output, and optionally gold answer, plus domain and difficulty. It should exclude anything that reveals the simulator's intent (category, `research_disagreement_type`, `target_disagreement_axis`) so raters stay blind. ⚠ confirm nothing like that leaks into `task["data"]` (see Q146).

q38. What is `hapax_ratio` in local scorer metrics? What does a high hapax ratio indicate about text quality? a38. The fraction of words that appear exactly once. High means varied vocabulary and little repetition, which is good for fluency. Extremely high on a long text can mean word salad, and on very short outputs it is always high. So I read it together with TTR.

q39. Explain the Prefilter Tradeoff Curve visualization. What do the X-axis, Y-axis 1, and Y-axis 2 represent? What does the optimal cutoff at 0.75 mean practically? a39. X is the local approval threshold τ. Y1 is cost reduction (share of probes handled locally). Y2 is decision safety (agreement preserved with the teacher on locally approved probes). Raising τ lowers savings and raises safety. At 0.75, about 65% of probes are handled locally while safety stays high. I would not quote "98%+ decision safety" from the old bullet unless I recompute it from the curve data, and the metric has a definitional flaw (Q77).

q40. Why does the Dockerfile run `python run_pipeline.py` during image build rather than at container startup? a40. So the image ships with data, metrics and figures already generated. Startup is fast, there is no compute at container start, and the artifacts are immutable. The cost is that a pipeline failure breaks the build, the data can go stale, and it only works for mock or offline mode (Q118).

q41. What is `use_column_width=True` in `st.image()` and why was it needed for Streamlit 1.65? a41. It stretches the image to the width of its column so the figure is responsive. I can't verify the specific "1.65" claim from the code I read. What I can say: newer Streamlit versions deprecate this argument in favour of `use_container_width`, so I would check the pinned version's docs. The app uses `use_column_width` for `st.image` and `use_container_width` for `st.dataframe`.

q42. In the agreement stats, why are Pearson r values computed between local scores and Rubric B specifically, rather than Rubric A? a42. Rubric B is the better-aligned judge and the one I would deploy, so the pre-filter has to predict that judge's decisions. Rubric A is a baseline. A stronger answer would also report local-vs-A to show the result is not rubric-specific.

q43. What is the `RubricConfig` Pydantic model used for? What happens if the `prompt_template` field is missing in a rubric YAML file? a43. It validates the rubric YAML at load time: name, axes, anchors, `prompt_template`, exemplars. If `prompt_template` is required and missing, Pydantic raises a `ValidationError` immediately, so it fails fast instead of a `KeyError` halfway through 200 evaluations. ⚠ confirm the field is required, not defaulted.

q44. Explain how the correlation matrix heatmap in `visualize.py` is constructed. Which 6 score dimensions are compared? a44. `plot_correlation_matrix_heatmap(human, rubric_b, local)` builds a table of scores across the three evaluators, computes a Pearson correlation matrix, and renders it with seaborn. ⚠ I can't state the exact six dimensions from memory. Open `visualize.py` and list them before the interview.

q45. What is `response_format={"type": "json_object"}` in the Groq API call? What problem does it solve? a45. An OpenAI-compatible option that constrains the model to emit a syntactically valid JSON object. It removes markdown fences, conversational chatter and most malformed JSON. It does not guarantee the right keys, types or ranges (Q65).

q46. The pipeline uses `.venv` isolation. Why is this critical for reproducibility on shared research computing environments vs. using system-level Python? a46. A per-project venv pins the dependency set independent of other users and projects, avoids version conflicts and pollution, and lets anyone rebuild the same environment from `requirements.txt`. The caveat: my requirements use `>=` ranges, so the environment is isolated but not exactly pinned (Q123).

q47. How does the `_extract_json()` method handle LLM responses that wrap JSON in markdown code fences? What regex pattern is used? a47. It is multi-stage. Try `json.loads` on the raw text. If that fails, use a regex for a fenced block, something like `` ```(?:json)?\s*(.*?)``` `` with `re.DOTALL`. If that fails, take the substring from the first `{` to the last `}`. ⚠ confirm the exact pattern in the code.

q48. What causes `UnicodeEncodeError: 'charmap' codec can't encode character '\u03ba'` on Windows, and how was it fixed? a48. The Windows console defaults to cp1252, which cannot encode κ (U+03BA) or Δ when printing. The fix is `sys.stdout.reconfigure(encoding="utf-8", errors="replace")` at the top of the CLI scripts (visible in `run_pipeline.py`). `PYTHONUTF8=1` is the environment-level alternative.

q49. The human annotation simulator uses different jitter ranges for `rubric_ambiguity` vs. `aligned` probes. What are these ranges, and why were they deliberately set differently?

a49. Aligned probes use small jitter (about ±0.10, per Q132) so the three annotators agree. Rubric-ambiguity probes use wider jitter so the annotators split and `avg_human_std` goes up. The point is to encode "the criterion is underspecified" as measurable human variance. ⚠ get the exact ambiguity range from the generator.

q50. What is the purpose of `use_container_width=True` in `st.dataframe()`? How does it affect the Streamlit probe inspector UI on different screen sizes? a50. It makes the table fill its container's width. In the two-column probe inspector it fits the narrower right column on small screens and expands on wide ones, avoiding horizontal scrolling or dead space.

HARD (Q51–Q100)

q51. The project's Fleiss's Kappa implementation takes an (N × k) ratings count matrix as input. Walk me through the complete mathematical derivation: what are $\bar{p}$, p_j, and p_e? a51. Let N be the number of items, n the raters per item (3), k the categories (4), and n_ij the number of raters who put item i in category j. Per-item agreement: $P_i = (1 / (n(n-1))) \cdot (\Sigma_j n_{ij}^2 - n)$. Mean observed agreement: $\bar{P} = (1/N) \Sigma_i P_i$. Category proportions: $p_j = (1/(N \cdot n)) \Sigma_i n_{ij}$. Chance agreement: $P_e = \Sigma_j p_j^2$. Then $\kappa = (\bar{P} - P_e) / (1 - P_e)$. It is the share of the above-chance agreement that was actually achieved.

q52. Quadratic Weighted Cohen's Kappa uses a weight matrix $w_{ij} = 1 - (i-j)^2/(k-1)^2$. For k=4 categories, compute the full 4×4 weight matrix and explain why off-by-two disagreements are penalized more than off-by-one. a52. With k = 4 the denominator is 9:

```
      0     1     2     3
0   1.000 0.889 0.556 0.000
1   0.889 1.000 0.889 0.556
2   0.556 0.889 1.000 0.889
3   0.000 0.556 0.889 1.000
```

The disagreement penalty is 1 − w: 1/9 for off-by-one, 4/9 for off-by-two, and 1 for off-by-three. So error cost grows with the square of the distance. A judge that says 0 when the human says 2 is penalised four times as much as one that is off by a single step, which matches how severity works on an ordinal scale.

q53. If two raters both give random uniform ratings across 4 categories, what is the theoretical expected value of Cohen's Kappa? Prove it mathematically. a53. Zero. For independent raters with uniform marginals, $P(\text{agree}) = \Sigma_j (1/4)(1/4) = 1/4$. The chance agreement is $p_e = \Sigma_j (1/4)(1/4) = 1/4$ as well. So $E[p_o] = p_e$ and the numerator $p_o - p_e$ has expectation 0, giving $\kappa \approx 0$. The same argument holds for the weighted version because both weighted observed and weighted expected agreement coincide in expectation. The sample κ is a ratio, so it has slight bias and variance of order 1/n, but its target value is 0.

q54. Why is $\kappa \geqslant 0.70$ the cited RLAIF viability threshold? What papers established this, and what assumptions does it make about the relationship between inter-annotator agreement

and downstream RLHF reward signal quality? a54. I don't know of a paper that established $\kappa \geq 0.70$ as an RLAIF threshold. It is a convention borrowed from annotation reliability: Landis–Koch's "substantial" band, and Krippendorff's 0.667 and 0.80 cut-offs. The supporting evidence for AI feedback comes from elsewhere: Constitutional AI (Bai et al., 2022), MT-Bench / LLM-as-a-judge (Zheng et al., 2023) showing GPT-4 agreeing with humans at about the human–human rate, and RLAIF-vs-RLHF comparisons (Lee et al., 2023). The assumption is that agreement with humans implies a good reward signal. That ignores correlated judge biases (verbosity, self-preference, position) that an RL policy can exploit. So "mathematically justifying RLAIF" in the README is an overclaim. "Meets a conventional reliability bar" is accurate.

q55. The project uses mock mode simulation where Rubric B adds +0.02 bias and Rubric A adds ±0.16–0.24 noise. How does this calibration decision affect the statistical validity of the Cohen's Kappa results? Is the simulation internally consistent? a55. The A→B improvement is built into the simulator by those parameters, so the result cannot be evidence that CoT rubrics improve alignment. It is internally consistent: more noise leads to more bin flips and lower κ, so the ordering is guaranteed. What it validates is the analysis pipeline, in that it can recover an injected effect. The scientific claim needs a live run with real annotation.

q56. The local scorer's Jaccard hallucination proxy achieves $r = 0.50$ Pearson correlation. Design an improved grounding metric using embeddings that could push this to $r \geq 0.80$ without requiring a 70B model call. What trade-offs does your design introduce? a56. Split the output into atomic claims (sentences or clauses). Embed claims and context sentences with a small encoder (MiniLM or bge-small), and take each claim's max cosine similarity to the context. Add a symbolic check: extract numbers, units, periods and named entities by regex, and require each to appear in the context. Embeddings alone barely distinguish 12.9 from 11.9, so the symbolic check is what makes the real gain. Score = min over claims. Optionally use a distilled NLI cross-encoder for contradictions. Trade-offs: latency rises from 0.05 ms to roughly 5–50 ms on CPU, a model dependency appears, thresholds need calibration against human labels, and 0.80 is a target, not a guarantee.

q57. A senior architect says: "Your Fleiss's Kappa calculation will be biased because your simulated annotators share the same base `true_quality` value from probe metadata. This creates artificial agreement inflation." How do you respond? Is this a valid criticism? a57. Valid. Annotators who are the same latent value plus independent jitter are far more consistent than real people, who have individual severity, bias and misreadings. So 0.829 is an optimistic ceiling. My response: label it as a simulated upper bound, add per-annotator bias and noise parameters to the simulator, and collect real labels for a subset through Label Studio to compare.

q58. The `compute_cohens_kappa()` function uses a normalized confusion matrix (divided by n). What numerical instability could arise when n is small (e.g., n=5 probes), and how would you guard against it in production? a58. With few items the confusion matrix is sparse, and p_o and p_e are very noisy. If either rater uses only one category, $1 - p_e \to 0$ and

κ is 0/0 or explodes. Guards: require a minimum n (for example ⩾ 30), return NaN with a warning rather than a made-up value, report bootstrap confidence intervals, and use prevalence-robust alternatives (Gwet's AC1, PABAK) when marginals are skewed.

q59. Explain the difference between the macro-level Fleiss's Kappa computed per-axis vs. an item-level agreement analysis. Which is more appropriate for this study and why? a59. Per-axis Fleiss κ is one chance-corrected reliability number per axis, aggregated over all items. Item-level analysis looks at agreement on each probe (P_i or the std of its three ratings) to find which specific items are contentious. This study needs both: macro κ for the human ceiling, and item-level variance (`avg_human_std`) to separate rubric ambiguity from judge error in the failure taxonomy.

q60. The pipeline generates probe categories with fixed `true_scores` for `judge_false_positive` (no_hallucination = 0.30) but the teacher judge scores it at 0.85. If you were writing a production-grade RLAIF system, how would you use this 0.55 delta to trigger a human review escalation policy? a60. In production there is no human label, so the 0.55 delta only exists offline. I'd use it to design online triggers that approximate it: escalate to human review when the judge says high groundedness (⩾ 0.7) but an independent signal disagrees (local grounding proxy low, numeric or entity mismatch against context, or a second judge differing by > 0.3). Log all escalations, sample some auto-approved items for audit, and use the confirmed false positives to tune the trigger thresholds.

q61. The Prefilter Tradeoff Curve shows Cost Reduction vs. Decision Safety. Derive the mathematical relationship between these two variables as a function of the threshold τ. At what τ does the marginal cost of escalation equal the marginal benefit of safety? a61. Let $A(\tau) = P(\text{local score} \geqslant \tau)$ be the approved fraction. Savings $S(\tau) = c \cdot A(\tau)$, where c is the cost per teacher call. Let $E(\tau) = P(\text{approved and the teacher would reject})$ be the unsafe-approval rate, with expected loss $L(\tau) = C_{err} \cdot E(\tau)$. Both A and E fall as τ rises. The optimum minimises $-S(\tau) + L(\tau)$, so at the optimum $c \cdot A'(\tau) = C_{err} \cdot E'(\tau)$: the marginal saving from lowering τ equals the marginal added expected error cost. The empirical curve puts that near 0.75 for this dataset, but the actual value depends entirely on the ratio C_{err} / c, which in finance is large and pushes τ up.

q62. The project's local scorer doesn't handle semantic similarity at all — only lexical overlap. Describe a scenario in financial QA where a semantically correct output has near-zero Jaccard overlap with the reference context, causing a false-negative hallucination flag. How would you architect around this? a62. Context: "Net interest income rose to $12.9 billion." Output: "NII increased to roughly thirteen billion dollars." It is correct, but almost no tokens overlap (abbreviation, number spelled out, rounding), so the local scorer flags poor grounding. Architecture: normalise numbers, units and finance abbreviations before comparing, use a small embedding or NLI check, and treat low lexical overlap as "send to teacher", never as an automatic reject.

q63. In the Streamlit dashboard, `@st.cache_data` caches `load_data()` across sessions. What happens if a new evaluation run updates `data/agreement_metrics.json` while the

app is running? How would you implement cache invalidation for this? a63. The app keeps serving the cached snapshot until the server restarts or the cache is cleared, so it shows stale metrics. Options: pass the files' modification times (or a content hash) as an argument to `load_data` so the cache key changes when files change, set a `ttl`, or call `st.cache_data.clear()` after a pipeline run. For production, read from a database or object store with a version key.

q64. Walk through the full Pydantic v2 validator chain when a model returns `{"fluency": "0.85"}` (string, not float). What coercions fire, in what order, and what is the final output? a64. Order in v2: `mode="before"` validators see the raw string, then core validation coerces the type (lax mode parses the numeric string "0.85" to float 0.85), then `mode="after"` validators run on the float. If `clamp_and_round` is an after-validator, it receives 0.85, finds it in range, and rounds. Final value 0.85. In strict mode the string would be rejected and trigger the re-query loop. ⚠ confirm which mode the validator uses.

q65. The `response_format={"type": "json_object"}` parameter forces JSON-only output from Groq. What failure modes remain that this parameter does NOT protect against? Name at least three. a65. (1) Wrong schema: missing or extra keys. (2) Wrong types or out-of-range values (95 instead of 0.95). (3) Truncation at `max_tokens` can still produce incomplete JSON. (4) Semantically wrong scores, or a score that contradicts the written reasoning. (5) Transport failures: 429 rate limits, timeouts, 5xx. (6) Prompt injection from the text being judged. None of these are fixed by JSON mode, which is why I validate with Pydantic and retry.

q66. You've deployed this on Streamlit Cloud with `run_pipeline(mock_mode=True, force_generate=True)` in the load path. What happens to cold-start latency, and what engineering change would you make for a production deployment with 1,000 concurrent users? a66. When files are missing, the first request runs the whole pipeline (about 3 seconds locally per the code comment, likely longer on throttled CPU because of figure rendering), blocking that user. With 1,000 users you risk a thundering herd of simultaneous regenerations. The fix is to generate artifacts at build time or in a separate job, version them, and have the app only read immutable files. Never run the pipeline in the request path.

q67. The project uses `random.Random(abs(hash(prompt + text[:25])) % (10**6))` to seed noise in Rubric A simulation. Why is this deterministic seeding important for research reproducibility? What could go wrong if Python's default `random` was used instead? a67. Seeding from the input means the same prompt and output always get the same simulated noise, so reruns match and tests are stable. With the global unseeded `random`, every run would give different scores and different κ. But note the flaw: Python's `hash()` on strings is randomised per process (`PYTHONHASHSEED`), so this is not deterministic across runs (Q95).

q68. Fleiss's Kappa assumes all subjects are rated by the same number of raters and ratings are independent. Which of these assumptions is violated in this project, and how does it affect the validity of the κ = 0.829 figure? a68. Equal raters per item holds (3 each). Independence is the problem: the simulated annotators share a latent value per probe, and

the items come from only 5 templates, so they are not independent draws. Real raters also share a rubric and training. Both effects inflate κ and make its confidence interval too narrow.

q69. The probe dataset uses only 5 financial templates repeated across 200 probes. From a statistical perspective, why does this template reuse inflate inter-rater agreement artificially? How would you redesign the dataset to avoid this confound? a69. Near-duplicate items are not independent, so the effective sample size is much smaller than 200, and raters (and judges) can learn template-specific cues, which inflates agreement. Redesign: use real passages from EDGAR filings or FinanceBench, diversify prompts and phrasings, stratify by domain and difficulty, de-duplicate near-matches, hold out a test set, and use cluster-aware confidence intervals.

q70. Design a production A/B testing framework to determine whether deploying Rubric B over Rubric A in a live RLHF reward model pipeline actually improves downstream LLM output quality. What are your control conditions, success metrics, and stopping criteria? a70. Control: reward labels from Rubric A. Treatment: Rubric B. Everything else is identical: same policy initialisation, prompts, RL budget, and at least 3 seeds each. Primary metric: blinded human or expert preference win-rate and hallucination rate on a held-out financial eval set (e.g., FinanceBench). Secondary: reward-model accuracy against human labels and judge–human κ. Guardrails: output length, refusal rate, KL drift, and reward-hacking indicators. Stopping: a pre-registered fixed budget or sequential test with alpha-spending, plus early stop on guardrail breach.

q71. The project achieves 0.048 ms per probe local evaluation. At what probe volume per day does this become the bottleneck versus network I/O to the 70B API, assuming a 250 ms API round-trip? Derive the crossover point. a71. Local capacity on one core is 1/0.048 ms ≈ 20,833 probes/s, or about 1.8 billion per day. A sequential API caller does 4/s, about 345,600 per day. With c concurrent in-flight calls the API side delivers c/0.25 s probes per second. Local becomes the bottleneck only when c/0.25 > 20,833, i.e. c > about 5,200 concurrent requests. Groq rate limits bind long before that, so local scoring is never the bottleneck in practice.

q72. The `compute_correlation_metrics()` function checks for constant arrays (all values equal) before computing Pearson r and returns 0.0, 1.0. Why is this check necessary? What would scipy.stats.pearsonr return for a constant input array in Python 3.11? a72. With a constant array the standard deviation is zero, so r = 0/0 is undefined. In recent SciPy versions `pearsonr` returns `nan` (statistic and p-value) and emits a `ConstantInputWarning`. The guard returns (0.0, 1.0) so NaNs don't propagate into the JSON report and heatmaps. The trade-off is that 0.0 reads as "no correlation" when the truth is "undefined". NaN with explicit handling would be more honest.

q73. Describe how you would implement a Bayesian version of Cohen's Kappa that produces uncertainty intervals around κ rather than a point estimate. Why might this be more useful in a research context? a73. Treat the k×k confusion matrix counts as

multinomial with a Dirichlet prior (uniform or Jeffreys). The posterior is Dirichlet(α + counts). Draw many posterior samples, compute κ for each draw, and report the posterior mean, a 95% credible interval and P(κ > 0.70). It is more useful because it quantifies uncertainty at small n and answers decision questions directly ("probability the judge clears the bar"), and a paired version gives P(κ_B > κ_A).

q74. The failure taxonomy shows 29.2% of disagreements are Judge False-Positives. In a live RLHF training loop, if the reward signal systematically over-rewards hallucinated outputs, what training failure mode does this induce? Name the specific phenomenon and its long-term effect on model behavior. a74. Reward hacking, specifically reward over-optimisation (Goodhart's law; see Gao et al., 2023). The policy learns to produce confident, fluent text that the judge scores highly whether or not it is true. Over training the proxy reward keeps rising while real factuality falls, so the model becomes confidently wrong, and the dashboards look fine while quality degrades.

q75. The project commits `data/` to git. What are the data engineering anti-patterns this creates at scale (e.g., 10M probes), and what infrastructure (DVC, S3, Artifact Registry) would you use instead? a75. Git is not built for large or binary data: repo bloat, slow clones, no deduplication, merge conflicts on JSON, no dataset lineage, and file-size limits. Instead use DVC (git-tracked pointers with an S3/GCS remote), object storage with Parquet, an artifact registry (MLflow, W&B Artifacts), and a lakehouse format (Delta or Iceberg) once volumes warrant it.

q76. Walk through the statistical test you would use to determine whether the improvement from Rubric A (κ=0.594) to Rubric B (κ=0.747) is statistically significant. What is the appropriate null hypothesis, test statistic, and required sample size for 80% power? a76. Both rubrics score the same 200 probes, so the test must be paired. H0: κ_B − κ_A = 0. The statistic is Δκ. Get its sampling distribution by bootstrapping probes (resample probes, recompute both κ, take the difference) and report the 95% CI, or run a permutation test that swaps A/B labels per probe. As a rough guide, κ's standard error at n = 200 is about 0.04, so a Δ of 0.15 is about 3 SE and would be significant. For 80% power on a 0.15 difference, roughly 120–150 items would do. To detect 0.05 you would need around 1,000+. Confirm by simulation. Note that significance on mock data only shows the simulator's injected effect is detectable.

q77. The `plot_prefilter_tradeoff_curve()` function defines "agreement preservation" as: if the local scorer approves an output AND the teacher judge also scores it ⩾ 0.70, it counts as preserved. What is the logical flaw in this definition, and how would you correct it? a77. It only credits approvals that happen to match the teacher and says nothing clear about the other three cells of the decision matrix. It also treats the teacher as ground truth instead of humans. It blends precision of approvals, wasted savings (teacher-good outputs the local scorer rejected) and correct rejections into one number. Correct it with a full confusion matrix of local decision vs gold decision. Report approval precision (safety),

false-approve rate and recall (savings) separately, against human labels, with confidence intervals.

q78. The hallucination proxy uses `min(0.92, max(0.20, jaccard_overlap * 1.5))`. Why is the ceiling set at 0.92 rather than 1.0? What implicit assumption about local evaluation reliability does this cap encode? a78. The cap encodes "a lexical proxy can never fully certify truth". Even perfect overlap leaves residual doubt, so the local score can never reach the top bins and cannot alone justify skipping the teacher on the hallucination axis. It is a conservative design choice, but it is a hard-coded magic number. It compresses the score range and caps the correlation. I'd calibrate it against human labels (e.g., isotonic regression) instead.

q79. Explain how entity-level hallucination (wrong numbers in financial statements) would score differently than omission hallucination (missing key figures) under the Jaccard overlap proxy. Which is more dangerous in a financial context, and how would you architect a detector for each? a79. A wrong number changes one token, so overlap stays high and the proxy barely moves. It misses exactly the dangerous error. Omission makes the output shorter than the context, so the union stays large and Jaccard drops, which means omissions get penalised even though nothing false was said. Entity-level errors are more dangerous because they are silent and confident. Detectors: for entities, extract numbers, units, periods and names, and require exact support in the context. For omissions, use a key-fact checklist from the gold answer or QA-based coverage and a recall-oriented metric.

q80. The project uses `scipy.stats.pearsonr` for correlation. Under what distributional assumptions is Pearson r the correct statistic here? Are these assumptions met given that scores are bounded in [0, 1]? What transformation would improve validity? a80. Pearson assumes a roughly linear relationship, approximately bivariate-normal data, homoscedasticity and no dominating outliers. Bounded, clustered scores with floors and ceilings (0.20, 0.92) and discretised levels violate these. Better: report Spearman or Kendall alongside, use a logit/probit transform before Pearson, and bootstrap the confidence intervals.

q81. A model generates: *"Q4 NII was $12.9 billion."* The reference context states: *"Net Interest Income was $12.9 billion."* The Jaccard overlap is very high (0.85), but the model hallucinated Q4 (it was actually Q3). How does your current pipeline handle this, and what would it take to catch it? a81. Currently the local scorer would approve it (high Jaccard). Only the teacher could catch it, and that is the judge-false-positive class (29.2% of disagreements). To catch it: extract temporal entities (Q1–Q4, FY, dates) and numbers from the output and require each to be supported by the context, so the unsupported "Q4" fails. Add an NLI check for unsupported claims, and put a period-swap example in the Rubric B exemplars.

q82. In the context of Constitutional AI and RLAIF at Anthropic, how does rubric-based critique-revision differ from the approach used in this project? What are the trade-offs in annotation quality, cost, and alignment consistency? a82. Constitutional AI (Bai et al., 2022)

has a model critique and revise its own outputs against written principles, then uses AI preference labels for RL. This project instead has an external teacher give pointwise 0–1 scores on four axes against human gold, with no revision step and no RL loop. Rubric scoring is easy to audit and measure with κ and gives anchored consistency, but it is a weaker training signal than pairwise preferences. CAI covers broader behaviours at similar cost, but principles can be vague. This project measures the judge, while CAI uses it.

q83. The project's disagreement threshold is $|\Delta| > 0.30$. How did you choose 0.30 vs. 0.25 or 0.35? Walk through the mathematical sensitivity analysis. What happens to the failure taxonomy distribution at each threshold? a83. Honestly, 0.30 was a practical choice: one Likert step is 0.33, so $|\Delta| > 0.30$ roughly means "at least one full step apart". I did not run a formal sensitivity analysis. To do it: rerun the analyzer at 0.20, 0.25, 0.30, 0.35, 0.40. The disagreement count falls monotonically with the threshold, and the category shares shift because the FP/FN and ambiguity rules depend on the same deltas. I would report the stable parts and flag the unstable ones.

q84. The self-healing `load_data()` in the Streamlit app calls `run_pipeline()` on cache miss. Explain the race condition that could occur with multiple simultaneous users hitting this path, and how you'd fix it using either file locking or a startup lifecycle hook. a84. Two sessions can both see missing files and both run the pipeline, writing the same JSON and PNG files concurrently. A third session can then read a half-written file. Fixes: wrap generation in a file lock (e.g., `filelock`), re-check file existence after acquiring it, and write to a temp file then `os.replace` for atomicity. Better still, generate before the app starts (build step or entrypoint).

q85. The pipeline writes scores to flat JSON. Design a scalable data model using PostgreSQL or a columnar format like Parquet that could handle 100K probes with efficient filtering by `category`, `domain`, `disagreement_type`, and axis score ranges. a85. Tidy long format. `probes(probe_id, prompt, context, output, gold, domain, category, difficulty, created_at)`. `annotations(probe_id, annotator_id, axis, score)`. `judge_scores(probe_id, run_id, rubric_version, axis, score, reasoning, model)`. `local_scores(probe_id, axis, score, latency_ms)`. Index on domain, category, and (axis, score), and partition by run_id. In Parquet, use the same tables partitioned by category and domain with predicate pushdown, and query them with DuckDB.

q86. What is the "alignment tax" in the context of RLHF, and how does this project's methodology help quantify it for financial domain LLMs? a86. The alignment tax is the capability regression a model suffers after alignment training (InstructGPT saw drops on some NLP benchmarks, which they mitigated with PPO-ptx). This project does not quantify it directly: it measures judge fidelity, not model capability before and after alignment. It helps only indirectly, by making the labels cheaper. To measure the tax you need before/after evaluations on a financial benchmark.

q87. The project generates 5 figures at 300 DPI. In a CI/CD pipeline, how would you implement automated visual regression testing to detect if a code change silently broke the

correlation scatter plot without a human reviewing the PNG? a87. Make rendering deterministic (fixed seed, Agg backend, pinned fonts and style), then compare against golden images with a tolerance, using `pytest-mpl` (`@pytest.mark.mpl_image_compare`) or an SSIM/pixel diff. Also assert on the underlying data (plotted arrays, regression slope, r) so a broken plot fails even when pixels move slightly. Store baselines in CI and fail on diff above the tolerance.

q88. The failure taxonomy categorizes Judge False-Negatives as the judge being "overly strict." In production, which failure mode (FP or FN) is more dangerous for a financial institution deploying an AI feedback system? Justify your answer with regulatory risk reasoning. a88. False positives are more dangerous. They silently approve or reward a wrong figure or fabricated claim, which becomes a misstatement risk in a regulated setting (liability, supervisory and recordkeeping obligations). False negatives mostly cost money and time (unnecessary rejection, rework), and they are visible and recoverable by human review. FN is not harmless, though: constant over-penalising can push a model to over-hedge. I'm not a lawyer, so I'd phrase the regulatory point as risk reasoning, not a legal claim.

q89. How would you extend this pipeline to a multi-judge ensemble (e.g., GPT-4, Claude 3.5, Gemini 1.5 Pro) and compute an aggregate agreement score? What consensus mechanism would you use, and how would you handle systematic per-model biases? a89. Run every judge with the same rubric and schema. Calibrate each judge against human labels (isotonic or z-score), then aggregate with the median or a validation-weighted mean, or take a majority vote on discretised bins. Measure agreement among judges with Krippendorff's α or Fleiss κ. For bias: exclude the candidate model's own family (self-preference), randomise ordering, control length, and weight judges by their κ on a human-labelled subset. Escalate to a human when judge variance exceeds a threshold. Cost scales with the number of judges.

q90. The Streamlit live playground evaluates custom text. If a user submits 10,000 characters of text, the TTR and n-gram calculations run client-side in Python synchronously. What is the UX impact, and how would you architect an async evaluation backend? a90. A small premise correction: Streamlit runs Python on the server, not the client. The TTR and n-gram work is $O(n)$ and takes milliseconds even at 10,000 characters, so it is not the real problem. The slow part would be the live teacher call (hundreds of ms to seconds), which blocks that session's script run. Architecture: a FastAPI service with a job queue (Celery or RQ), the UI polling or using WebSocket updates, results cached by text hash, plus input length limits and a spinner.

q91. If you had to reduce the Groq 70B teacher evaluation cost by 90% while maintaining $\kappa \geq 0.70$ alignment, describe a knowledge distillation approach that trains a smaller judge model on Rubric B outputs. What training data, loss function, and evaluation protocol would you use? a91. Student: a small model (an 8B LLM or a DeBERTa-class encoder). Data: tens of thousands of diverse, real (prompt, context, output) triples labelled by the 70B

teacher with Rubric B, plus a human-labelled calibration subset. Loss: ordinal cross-entropy or Huber regression on the four scores, optionally a token-level SFT loss for the rationale. Evaluation: κ against humans on a held-out set, κ against the teacher, per-axis results, calibration (ECE), and latency and cost. Deploy as a cascade: the student answers when confident, and low-confidence cases escalate to the 70B teacher. Accept only if κ vs humans stays $\geqslant 0.70$.

q92. The project uses `seaborn` for the correlation heatmap. In a production BI environment for a bank, why would you replace this with an interactive Plotly or Vega-Altair chart, and what specific financial analyst workflows does interactivity enable? a92. Interactivity lets analysts hover to see the probe ID and score, filter by domain or category, click an outlier to open the underlying text, adjust the threshold and see the tradeoff curve update, and export the filtered rows. That supports disagreement triage, threshold tuning and audit evidence, none of which a static PNG allows.

q93. Explain transfer learning degradation: if you train a reward model on this project's 200 probe dataset and deploy it on a different financial LLM benchmark (e.g., FinanceBench), what distribution shift failure modes would you anticipate? a93. Covariate shift: FinanceBench has long real filings, tables and multi-step numeric reasoning, and my probes are short template text. The reward model would likely have learned template cues (shortcut learning). Label shift: different base rates of hallucination. Calibration collapse on unfamiliar formats, and failures on numeric reasoning it never saw. Mitigation: diverse training data, held-out evaluation on the target benchmark, and uncertainty-aware routing. This project doesn't train a reward model; this is how I'd expect one to behave.

q94. The rubric YAML files define `axes` as a list of dicts. Design a JSON Schema that validates a rubric YAML file and catches: missing required prompt_template, invalid axis names, and score anchor values outside [0, 1]. a94.

```json
```

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "required": ["name", "prompt_template", "axes"],
  "additionalProperties": false,
  "properties": {
    "name": {"type": "string"},
    "prompt_template": {"type": "string", "minLength": 20,
      "pattern": "\\{model_output\\}"},
    "axes": {
      "type": "array", "minItems": 1,
      "items": {
        "type": "object",
        "required": ["name", "anchors"],
        "properties": {
          "name": {"enum": ["fluency", "coherence",
                            "instruction_adherence", "no_hallucination"]},
          "description": {"type": "string"},
          "anchors": {
            "type": "object",
            "additionalProperties": {"type": "string"},
            "propertyNames": {"pattern": "^(0(\\.\\d+)?|1(\\.0+)?)$"}
          }
        }
      }
    }
  }
}
```

It rejects a missing `prompt_template`, any axis name outside the four, and anchor keys outside [0, 1]. Load the YAML to a dict and validate with `jsonschema`.

q95. The project's mock teacher judge uses deterministic hashing to seed noise: `abs(hash(prompt + text[:25]))`. On Python 3.3+, `hash()` is randomized by default with `PYTHONHASHSEED`. How does this affect reproducibility, and what change would you make?
a95. If the code is as the question describes, it breaks reproducibility. String hashes change every process, so the mock noise (and therefore κ and the failure counts) can differ run to run. The headline numbers would then be one draw. Fix: derive the seed from a stable hash such as `int(hashlib.sha256(s.encode()).hexdigest(), 16) % 10**6` (or `zlib.crc32`), and as a stopgap set `PYTHONHASHSEED=0` in the Dockerfile and test environment. ⚠ check whether the 18 tests assert exact numbers, since they would be flaky under this bug.

q96. The project achieves κ = 0.829 human ceiling and κ = 0.747 teacher alignment. Calculate the ceiling-relative alignment efficiency: what percentage of maximum

achievable agreement does the teacher judge reach? How does this compare to the literature benchmarks for LLM-as-a-Judge alignment? a96. 0.747 / 0.829 = 90.1%. Caveats: the ceiling is simulated, and one number is quadratic-weighted Cohen while the other is Fleiss, so the ratio is only indicative. In the literature, Zheng et al. (2023) report GPT-4 agreeing with humans at about the level humans agree with each other (roughly 80–85% on MT-Bench). That is percent agreement on a different task, so a direct comparison with my κ would be invalid. Being "near the human ceiling" is consistent with that finding, nothing more.

q97. Walk through the `compute_fleiss_kappa()` mathematical derivation for the case where all 3 annotators always agree. Prove algebraically that $\kappa = 1.0$. Then prove it approaches a lower bound when raters assign ratings randomly. a97. All agree: for each item one category has $n_{ij} = n = 3$ and the rest are 0. $P_i = (3^2 - 3) / (3 \cdot 2) = 1$, so $\bar{P} = 1$ and $\kappa = (1 - P_e) / (1 - P_e) = 1$, provided $P_e < 1$. Random raters, uniform over k categories: the chance two raters agree is 1/k, so $\bar{P} \to 1/k$, and $p_j = 1/k$ gives $P_e = \Sigma (1/k)^2 = 1/k$. So $\kappa = 0$. Random ratings give $\kappa \approx 0$, not the lower bound. The true lower bound is negative and comes from systematic disagreement: for $n = 3$, $k = 4$, if all three raters always differ, $\bar{P} = 0$ and $\kappa = -P_e / (1 - P_e) = -1/3$ when $P_e = 1/4$.

q98. The project's failure analysis shows 48 total disagreements out of 200 probes (24%). Calculate the 95% confidence interval for this disagreement rate using the Wilson score interval method. Is the sample size sufficient to detect a 5% difference in disagreement rates between rubrics? a98. $\hat{p} = 0.24$, $n = 200$, $z = 1.96$. Wilson centre = $(\hat{p} + z^2/2n) / (1 + z^2/n) = 0.2490 / 1.0192 \approx 0.245$, and the half-width ≈ 0.059, so the 95% CI is about **[18.6%, 30.4%]**. For detecting a 5-point difference (say 24% vs 29%) between two independent groups at 80% power, you need about 1,200 probes per group, so 200 is not enough. A paired McNemar design needs fewer but still likely several hundred. Also, my 48 disagreements are computed for Rubric B only, so no rubric-to-rubric comparison exists yet.

q99. Design an adversarial probe generation strategy that specifically targets the weakest axis in this study (No Hallucination, $r = 0.500$). What prompt patterns would you use to systematically generate probes that expose local scorer false-negatives? a99. Generate minimal-edit counterfactuals from correct gold answers so overlap stays near 1.0 while the truth changes: number and unit swaps, period swaps (Q3→Q4), entity and segment swaps, and negation flips ("increased" → "decreased"). Add plausible unsupported additions ("driven by strong consumer demand") and fluent fabricated explanations. Also generate the reverse: heavy paraphrase that is correct but has low overlap. Label each probe with its edit type, verify with a human, and track the local scorer's miss rate per type to guide a better proxy.

q100. The project is structured as a research study. Convert it to a production-grade financial AI quality gate deployed in a CI/CD pipeline that runs every time a new LLM fine-tune is pushed. What SLAs, thresholds, alerting, and rollback mechanisms would you define? a100. Trigger on each pushed checkpoint. Run a frozen, versioned held-out set (⩾

1,000 real probes) with deterministic settings: local pre-filter, then the teacher on escalated items and always on the hallucination axis. Gate thresholds: no-hallucination must not fall below baseline minus ε, hard fail above a set hallucination rate, with floors on fluency and adherence. Judge health check: a weekly audit sample against human labels must keep $\kappa \geqslant 0.70$, otherwise the gate is flagged unreliable. SLAs: gate completes within about 15 minutes, p95 judge latency target, API error budget with a fallback model, and a cost cap. Alerts via Slack or PagerDuty on failure or drift. Rollback: a failing gate blocks promotion in the model registry, the production alias reverts to the last passing model, and releases go out as a canary with automatic rollback on live metrics. Borderline cases go to human review.

TRICKY KNOWLEDGE-BASED (Q101–Q150)

q101. The project reports "Fleiss's Kappa = 0.829 across 3 human annotators" but the annotators are simulated, not real humans. Does this make the κ value meaningless? When is simulated IAA analysis still scientifically valid? a101. Not meaningless, but it is not evidence about real annotators. Simulated IAA is valid for validating the measurement code (can it recover a known injected κ?), power and sample-size planning, stress-testing metrics under assumed noise, and sensitivity analysis, especially when simulator parameters are calibrated from a real pilot. It is not valid as a claim about the real human ceiling. I present 0.829 as a simulated reference value.

q102. Cohen's Kappa corrects for chance agreement. If two raters always agree but always assign the same score to everything (e.g., both rate everything 3/3), what is Cohen's Kappa? Is this a good outcome? a102. $p_o = 1$ and $p_e = 1$, so $\kappa = 0/0$, which is undefined (some implementations return 1.0 by convention, as mine does for the edge case). It is not a good outcome: there is no variation, so the ratings carry no information about discrimination. It is the "kappa paradox", where prevalence skew breaks κ. Use PABAK or Gwet's AC1, and make sure the data covers the full quality range.

q103. The rubric uses "No Hallucination" as an axis. Strictly speaking, can any statistical metric prove the *absence* of hallucination, or does it only measure *absence of detected* hallucination? What are the epistemological implications for deploying this as a safety gate? a103. It can only measure absence of detected hallucination: you can falsify "this output is grounded" with an error, but you cannot verify it exhaustively. So the gate gives risk reduction with a measurable miss rate, not a guarantee. Implications: report the detector's recall with confidence intervals, keep conservative thresholds and human review for high-stakes outputs, monitor in production, and name the axis "groundedness score", not "no hallucination".

q104. A colleague says: "Your Pearson r = 0.820 on fluency proves the local scorer can replace the 70B teacher." What is the critical flaw in this reasoning? a104. Correlation is not agreement or decision quality. r = 0.82 can coexist with offsets, scale differences, and misses on exactly the rare bad outputs that matter. It holds for one axis on 200 template-generated probes, and it is measured against the teacher, not humans. The same scorer has r = 0.50 on

hallucination, 0.365 on coherence and 0.234 on adherence, so it cannot replace the teacher. What it supports is a narrow claim: fast-pass triage for fluency. To claim more you need precision and recall at the decision threshold against human gold.

q105. The project's classification puts "Rubric Ambiguity" at 25.0% of all disagreements. A skeptic argues this number is unfalsifiable since you defined the categories yourself. How do you defend the taxonomy's objectivity? a105. The categories have explicit operational rules (thresholds on delta, judge score, human std), so the classification is reproducible and testable. I'd defend it three ways: have two humans independently label a sample of the 48 cases and compute κ against the automatic taxonomy, run a threshold sensitivity analysis, and in simulation measure how well the classifier recovers the injected `research_disagreement_type` with a confusion matrix. I'd also admit that in mock mode the 25% is partly set by how many ambiguity probes I generated.

q106. The mock judge gives `judge_false_positive` cases a hallucination score of 0.85 while humans rate 0.30. This is baked into the simulation. Does this mean the failure analysis results are circular/predetermined, or is there still scientific value? a106. The category counts are partly predetermined, so it is partly circular. The value is in testing the analyzer: does the rule-based classifier correctly recover injected failure types, how sensitive is it to thresholds, and does the pipeline run end to end? It gives a template to run on real judge outputs. What it cannot support is the claim that real judges fail in these proportions.

q107. The project claims 65% API cost reduction via fast-pass filtering. But the local scorer runs on EVERY probe, including the 35% that get escalated to 70B anyway. Does this change the cost savings calculation? Compute the correct figure. a107. Barely. The local scorer costs 0.048 ms of CPU per probe and no API spend. Average latency per probe = $0.048 + 0.35 \times 1200 = 420.05$ ms against 1,200 ms, a 65.0% reduction, so the overhead is about 0.004%. The deeper problem is consistency: one teacher call returns all four axes, and the policy says the hallucination axis always escalates. If that holds, no probe can skip the teacher and savings approach zero. The 65% only holds if some probes are approved without a hallucination check, which is the unsafe part (Q143). So treat 65% as a policy-dependent estimate.

q108. The `LightweightStudentEvaluator` filters common stopwords before computing Jaccard. Name three financial domain terms that are high-frequency across all financial documents and would be incorrectly treated as stopwords by a generic English stopword list. a108. There are two separate issues. Generic lists (e.g., NLTK's) wrongly remove directional and negation words that carry meaning in finance: **up/down, above/below, over/under, no/not** ("did not exceed", "up 3%"). Dropping them makes opposite statements look identical. Conversely, finance-frequent words like **net, total, million/billion, percent** are not in generic lists, so they are kept and inflate overlap between unrelated sentences. Fix: use a domain-aware stopword list or TF-IDF weighting. ⚠ check which list the scorer uses.

q109. The project asserts Rubric B "CoT + Exemplars" is better than Rubric A "Zero-Shot." Is this a fair comparison? What confounds exist (temperature, token length, exemplar quality) that make the A/B rubric comparison not fully controlled? a109. Not fully. B differs from A in several things at once: anchors, CoT, exemplars, output length and possibly format. Exemplars may resemble the test templates (leakage), temperature and seeds may differ, and there is a single run. In mock mode the difference is just a parameter. A fair comparison needs an ablation grid (anchors only, CoT only, exemplars only), temperature 0, exemplars held out from the test probes, multiple seeds, and bootstrap CIs.

q110. The pipeline uses Python's `random.Random(seed)` for deterministic probe generation. Why is this NOT cryptographically secure and why does it matter (or not matter) in this research context? a110. It uses the Mersenne Twister, whose state can be reconstructed from about 624 outputs, so it is predictable and unsuitable for secrets (use the `secrets` module for that). It doesn't matter here because there is no adversary and no secret: the goal is a reproducible, statistically reasonable stream.

q111. A probe's `ground_truth_quality` is set by the data generator, not real humans. If this dataset were submitted to a peer-reviewed NLP conference, what reviewers' objection would you anticipate first? a111. Circularity: the "ground truth" and the simulated annotators come from the same generator that produced the outputs, so the study measures the simulator, not the judge. Next would come: no real human annotation, only 5 templates, a mock judge in the main results, no significance tests or confidence intervals, and no comparison to existing benchmarks. I'd pre-empt it by clearly labelling the study as a methodology and pipeline demonstration and by running a real annotated subset.

q112. The Streamlit app uses `@st.cache_data` on `load_data()`. This means data is cached for the lifetime of the server session. If 5 users open the app simultaneously for the first time, how many times does `run_pipeline()` execute? a112. Likely zero, because `data/` and `figures/` are committed, and `load_data` only calls the pipeline when files are missing. If they were missing, I believe Streamlit serialises concurrent calls with the same cache key so the first computes and the others wait (one execution). I'd verify that for the pinned version. If it does not lock, up to five executions could race and write the same files (Q84).

q113. The project achieved "18/18 tests passing." Does 100% test pass rate mean the system is correct? What category of bugs does the test suite NOT catch? a113. No. Passing tests mean the code does what the tests assert. They don't catch: statistical or scientific validity problems, simulation circularity, threshold choices, hash-randomisation non-determinism, live-API behaviour (tests mostly use mock mode), concurrency races, UI regressions, Windows-only encoding issues, and drift when dependencies change. Eighteen tests also cover a small slice of 7+ modules.

q114. The project uses `min(0.92, ...)` as a ceiling for the hallucination proxy. This means no probe can ever receive a perfect 1.0 grounding score locally. Is this a bug or a feature? Justify your answer. a114. A defensible feature with a flaw. It encodes that a lexical proxy

can't certify truth, so local scores never reach the "fully verified" range and escalation stays possible. The flaw is that it is an unexplained magic constant that compresses the range and limits correlation. I would keep the principle, make the cap configurable, and calibrate it against human labels.

q115. The `compute_cohens_kappa()` function handles an edge case where `1 - p_e ≈ 0` and returns 1.0. Is this mathematically correct? What real-world scenario produces p_e ≈ 1.0 and what does κ = 1.0 mean in that context? a115. Not strictly correct. When p_e = 1 the formula is 0/0, and returning 1.0 is a convention (perfect observed agreement). It happens when both raters use a single category for every item, for example every probe scored 3. Then κ = 1.0 only means "no disagreement was observed", and it carries no information about discrimination. NaN with a warning, plus p_o reported alongside, is more honest.

q116. The project classifies "Judge False-Negative" as: judge scored low, human scored high. But in a domain where human annotators are known to be overgenerous (common in financial compliance), who is actually "correct" — the strict judge or the lenient human? a116. Neither is correct by definition. Correctness needs an external reference. For factual claims there is one: check the output against the source document. So adjudicate the FN cases with a senior expert panel against the source, and treat the human consensus as a noisy label. If humans are systematically lenient, judge–human κ understates the judge's validity. I'd report agreement against the adjudicated gold on the disputed cases too.

q117. The pipeline auto-generates 200 probes deterministically using `seed=42`. A researcher runs the pipeline twice on the same machine and gets identical results. Does this mean the results are reproducible? What happens when run on a different Python version? a117. Only weakly. Identical runs on one machine show repeatability in one environment, not reproducibility. Across Python versions, the Mersenne Twister stream itself is stable, but derived methods (`choice`, `sample`, `shuffle`) have changed behaviour across releases, and numpy/scipy/matplotlib versions can change numbers and styles. And if the mock judge seeds from `hash()`, results differ across runs, even on one machine (Q95). So I'd pin versions, use a stable hash, record the environment in output metadata, and ship it in a locked Docker image.

q118. The Dockerfile runs `python run_pipeline.py` during image build. If the pipeline fails during build, the Docker image build fails. Is this good or bad practice for a production deployment, and what alternative would you use? a118. Mixed. Good: fail-fast, and artifacts are baked and immutable. Bad: data generation is coupled to the image build, builds get slow, data goes stale, a pipeline problem blocks unrelated deploys, and live-API secrets aren't available at build time. Better: run the pipeline as a separate CI or scheduled job that publishes versioned artifacts to object storage. The image holds only app code and pulls the artifacts at startup, and the pipeline's tests run in CI.

q119. A Groq API call fails on attempt 1 and 2 due to rate limiting (HTTP 429), then succeeds on attempt 3. The retry loop feeds validation errors back to the model. Is it appropriate to send validation error messages to the API for rate-limit failures? What should the retry logic

differentiate? a119. No. A 429 has no response content to validate, so there is no validation error to send. The logic should separate failure types: transport errors (429, 5xx, timeouts) get exponential backoff with jitter and honour `Retry-After` with an unchanged prompt, while content errors (invalid JSON, out-of-range, schema mismatch) get the error-feedback re-query. Keep separate counters, add a circuit breaker, and consider a fallback model.

q120. The agreement bar chart shows Rubric B outperforms Rubric A on Fluency and Coherence but underperforms on Instruction Adherence. If you were presenting this to a CTO, would you report "Rubric B is better" or "results are mixed"? What does the overall κ number hide? a120. "Mixed, with a net gain." Per the README table: fluency +0.052, coherence +0.098, adherence −0.038, hallucination +0.011. The overall figure hides that adherence got worse and the hallucination axis, the one that matters most, barely moved. And the overall number itself needs checking: Rubric A's per-axis values average 0.717, not the reported 0.594, and `OptimizationFix.md` gives different A values for fluency (0.466) and coherence (0.541). Until that is reconciled against `agreement_metrics.json`, I'd quote only Rubric B's per-axis numbers and be careful with "+0.153".

q121. The project uses Streamlit for the dashboard. A hedge fund CTO says: "We can't use Streamlit in production — it's not enterprise-grade." What are the legitimate technical concerns, and what would you migrate to? a121. Legitimate concerns: the script-rerun model with per-session state in server memory doesn't scale well, there is no built-in SSO/RBAC or fine-grained authorisation in the open-source version, limited audit logging and observability, and heavy computation blocks sessions. For production I would put a FastAPI backend behind a React front end (or Dash/Panel), or use the bank's BI stack (Superset, Tableau, Power BI) for analytics, keeping Streamlit for internal prototypes.

q122. The label `"research_disagreement_type"` is in every probe record. This field drives all the simulation logic. If this field were removed from production records, what would happen to the teacher judge's accuracy claims? a122. In mock mode, the judge would default every probe to "aligned", so it would look more accurate, and the FP/FN categories would vanish. The claims about failure proportions are properties of the simulator, not of Groq. In live mode the field is unused, so real accuracy would be whatever the model actually does, and the reported numbers say nothing about it until a `--live` run is analysed.

q123. The project's `requirements.txt` pins `pydantic>=2.0.0` rather than an exact version. In a production ML pipeline, why is this a risk, and what would you do differently? a123. Unpinned ranges let builds drift. A new Pydantic, SciPy, Streamlit or NumPy release can change validator behaviour, add deprecations (like `use_column_width`), or alter edge cases (SciPy's constant-input handling), so the same code gives different results over time. I'd generate a lockfile (pip-tools, uv or poetry) with hashes, pin the base image by digest, use Dependabot and a CI matrix to upgrade deliberately.

q124. The project's correlation analysis uses all 200 probes including the simulated disagreement cases. Would removing the 48 disagreement probes increase or decrease the

Pearson r between local and teacher scores? Why? a124. It could go either way, since two effects compete. If those probes add noise (the mock teacher scores FP cases 0.85 while the true quality is 0.30), removing them raises r, especially on the hallucination axis. If they are the extreme-quality cases, removing them restricts the range and lowers r. I'd expect hallucination r to go up and fluency to change little, but I'd recompute to confirm. Also removing 24% of data by outcome introduces selection bias.

q125. The `figures/` directory is committed to git. GitHub has a 100MB file size limit and LFS recommendations for binary files. What CI/CD issue would appear at scale if you generate 1,000 figures per experiment? a125. Repository bloat: hundreds of MB to GB of PNG history that never compresses away, slow clones and CI checkouts, binary merge conflicts, and push-size limits. The 100 MB per-file limit is not the issue, total repo size is. Generate figures in CI as build artifacts or push them to object storage or Git LFS or DVC, and keep them out of git history.

q126. The hallucination axis has the highest Fleiss's $\kappa = 0.951$ among human annotators but only $r = 0.500$ correlation with the local scorer. Explain this apparent contradiction — high human agreement + low local correlation. a126. There is no contradiction, because the two numbers measure different things. High κ means the label is well-defined and stable across humans, who can check a number against the context. The $r = 0.50$ says the lexical Jaccard proxy is a poor measure of that well-defined construct. Reliability of the label does not imply validity of the cheap proxy. Also, $\kappa = 0.951$ is inflated by the shared latent value in the simulated annotators.

q127. The Streamlit live playground evaluates text using `mock_mode=True`. A user pastes a real 10-K excerpt and asks "Is this hallucinated?" What is the correct answer the system gives, and is it accurate? a127. The system gives a number, but it isn't meaningful. The "teacher judge" in the playground is the mock simulator, which doesn't read the text semantically. It produces seeded pseudo-scores from the input. The local grounding score is only token overlap. So no, it is not an accurate hallucination check. I'd add a visible "SIMULATED" label to the playground, and rename the panel so it doesn't imply a real 70B judge.

q128. The project stores probe IDs as `probe_001`, `probe_002`, etc. At 1 million probes, what indexing problem does this string format create in a database, and what ID scheme would you use instead? a128. Zero-padded string IDs sort lexically, so they break past the padding width (`probe_1000000` sorts before `probe_999999`), and string primary keys make larger, slower indexes than integers. Use a BIGINT identity or UUIDv7/ULID (time-ordered, globally unique and index-friendly), and keep any readable label as a separate column. A content hash of the prompt and output can serve as a dedupe key.

q129. The `run_pipeline.py` has a `force_generate=False` default. When the Streamlit app calls `run_pipeline(force_generate=True)`, it regenerates all 200 probes from scratch. Why is this problematic for a deployed app where users may be mid-session? a129. It overwrites the data files under active users. One session may hold cached old data while

another reads new files or a half-written file, so metrics and figures can mismatch. If the generator isn't truly deterministic, the numbers change between reloads. It is also heavy recomputation inside a request. Use versioned immutable artifacts with an atomic swap, and run generation as a background job, never in the user path.

q130. The failure taxonomy uses `avg_human_std > 0.65` as the trigger for "Rubric Ambiguity." This threshold was chosen manually. What data-driven method would you use to learn the optimal classification boundary? a130. Get expert-assigned causes for a sample of disagreements (ambiguity vs judge error). Then either run an ROC analysis on `avg_human_std` and pick the cut-off by Youden's J or by a cost-weighted criterion, or fit a small classifier (logistic regression or a shallow tree) on features like human std, delta, signed delta, axis and text length, with cross-validation. If the std distribution is bimodal, a Gaussian mixture can also locate the boundary. Check stability with bootstrapping.

q131. The project structure has `annotation/`, `analysis/`, `evaluation/`, and `scripts/` as separate directories. A new engineer asks: "Why not put everything in `src/`?" What is your architectural justification for the current layout? a131. The layout separates concerns by pipeline stage: annotation (data ingestion), evaluation (local scorer and teacher), analysis (stats and plots), scripts (data generation), so each stage is independently testable and could become its own package or service. Flat top-level packages also import easily without installing. But the new engineer has a point: a `src/judgellm/...` layout with a `pyproject.toml` is better practice for packaging, avoids accidentally importing from the working directory, and supports `pip install -e .`. If this became a shared library, I'd migrate.

q132. The project uses `rng.uniform(-0.10, 0.10)` jitter for aligned probes. This means aligned probes can still occasionally show a delta > 0.30 if scores are near the boundary (e.g., human = 0.85, teacher = 0.75, then with jitter human could be 0.79 and teacher 0.87). Does this create false disagreements? How frequent would they be? a132. The example doesn't work: those deltas are 0.10 and 0.08, nowhere near 0.30. If both sides get independent ±0.10 jitter around the same value, the difference is triangular on [−0.20, 0.20], which can't exceed 0.30. False disagreements can arise only by stacking Likert quantisation (steps of 0.33) and the +0.02 bias on top, and that should be rare. Also, "disagreement" is counted over four axes, which multiplies the chances. The right way to settle it is a Monte Carlo run of the generator on aligned probes only.

q133. A senior researcher points out: "Your Cohen's Kappa computation discretizes continuous scores to 4 bins, losing information. You should use a continuous agreement metric like Lin's Concordance Correlation Coefficient (CCC) instead." How do you respond? a133. Fair point, and I'd partly agree. I discretise because the human labels are ordinal 0–3 Likert, κ is the standard, literature-comparable metric for that, and the downstream decisions are threshold-based. Quadratic-weighted κ is also closely related to the ICC (Fleiss & Cohen, 1973), which limits the information loss. But CCC is better for the continuous judge scores against the continuous consensus mean, because it penalises both

scale and location shifts. I'd report both, plus Spearman, with bootstrap confidence intervals.

q134. The project deployed on Streamlit Cloud with the self-healing pipeline. On Streamlit Community Cloud, apps are put to "sleep" after inactivity. When a user wakes the app, `load_data()` cache is cleared. What happens on first load, and how long does it take? a134. On wake the container restarts, imports load, and the cache is empty. Because `data/` and `figures/` are committed, the files exist and `load_data` just reads about seven JSON files (well under a second), so the pipeline doesn't run. Wake time is dominated by container startup. If the files weren't committed, it would run the pipeline, about 3 seconds locally and probably longer on shared CPU. I can't give an exact figure without measuring, so I'd time a cold start.

q135. The prompt template in rubric_b.yaml uses `{model_output}`, `{prompt}`, and `{context}` placeholders. If a financial model output contains the literal string `{context}`, what bug appears, and how do you fix it safely? a135. It depends on how the template is filled. With chained `.replace("{context}", ...)` calls, a `{context}` string inside the model output is replaced by the context text, corrupting the evaluated content and opening a prompt-injection path. `str.format` doesn't re-parse substituted values, but it fails on literal braces in the template itself (such as an inline JSON example) with a `KeyError`. Safe fix: single-pass substitution (`string.Template`, or a regex substituting all placeholders at once), or pass content as separate delimited message fields instead of interpolating it into one string, and escape template braces. ⚠ check how `teacher_judge.py` fills the template.

q136. The project's `compute_fleiss_kappa()` processes an (N × k) matrix where k is fixed at 4 categories (0–3 Likert). What happens mathematically if one category (e.g., category 0) is never assigned by any annotator? Does Fleiss's Kappa still return a valid result? a136. Yes. An unused category has p_j = 0, so it adds 0 to P_e and nothing to P, and the formula works as if there were three categories. It breaks only if everything lands in one category, since then P_e = 1 and κ is 0/0.

q137. The project uses `seaborn-v0_8-whitegrid` style with a version-safe fallback to `"default"`. In a containerized environment using `matplotlib==3.11.2`, which style name would be active? How would you verify this without running the code? a137. I can't confirm that specific matplotlib release. The `seaborn-v0_8-*` style names have been the valid ones since matplotlib 3.6 (the old unprefixed `seaborn-*` names were removed), so in any 3.6+ release where they are still shipped the primary name is active and the `"default"` fallback fires only on a missing style. Without running the pipeline you can check the installed wheel's `matplotlib/mpl-data/stylelib/` directory for `seaborn-v0_8-whitegrid.mplstyle`, or inspect the package contents on PyPI, or in a one-liner check `'seaborn-v0_8-whitegrid' in plt.style.available`.

q138. The `FailureModeAnalyzer` finds the "primary axis" of disagreement by taking `max(disagreed_axes, key=lambda a: disagreed_axes[a]["delta"])`. What happens if

two axes have identical deltas? Is the resulting classification stable or arbitrary? a138. Python's `max` returns the first maximal element in iteration order. Dicts keep insertion order, so the result is deterministic for a given axis order, but it is arbitrary in meaning. Reordering the axes would change the primary axis and could change the category. Fix: add an explicit tie-break (for example a priority order with no-hallucination first, or the larger human std), or record all tied axes.

q139. The `_mock_evaluate()` method reads `metadata.get("research_disagreement_type", "aligned")`. In production with real Groq API calls, this metadata is never used. Explain precisely why the mock and live code paths are architecturally separate, and what risk this separation introduces. a139. They are separate so tests, demos and the deployed app run offline with no API key, no cost and no nondeterminism, and the mock needs the scenario label to simulate behaviour. The live path sends only the prompt, context and output. Risks: the live path is under-tested (prompt formatting, retries, parsing), mock output can be mistaken for real evidence, mock and live output schemas can drift, and the metadata could leak into a live prompt by accident. Mitigations: contract tests that both paths pass, recorded real responses as fixtures, a nightly `--live` smoke test, and a `judge_mode: mock|live` field in every output file plus a visible dashboard banner.

q140. The agreement stats compute "overall" as the arithmetic mean of per-axis κ values. Is an arithmetic mean the correct aggregation for Kappa statistics? What would a weighted mean (weighted by axis prevalence) look like, and when would it produce a different result? a140. It is common but not strictly right: κ is not on a ratio scale, and each axis has a different chance-agreement level. Alternatives are a pooled κ (one confusion matrix over all axis ratings) or a weighted mean by importance or prevalence. For example, weighting no-hallucination at 0.4 and the others at 0.2 gives Rubric B ≈ 0.757 versus 0.747 unweighted, because that axis is the highest. One more thing to check: Rubric B's per-axis values do average exactly to 0.747, which confirms "overall" is the arithmetic mean. But Rubric A's README values average 0.717, not 0.594, so one of the two is wrong (Q120).

q141. The project reports the self-healing pipeline runs in ~3 seconds on a fresh deployment. On Streamlit Community Cloud, CPU is throttled. What realistic cold-start time estimate would you provide to a user, and what UX affordance would you add? a141. I haven't measured it there. The 3 seconds is a local figure, and figure rendering at 300 DPI probably dominates, so on shared throttled CPU I'd expect a multiple of that (order of 10–30 seconds if the pipeline actually runs). I'd measure it before promising a number. UX: a spinner with a clear message ("first load generates the demo data"), serve precomputed artifacts so the pipeline rarely runs, and a status note about wake-up delay.

q142. The `OptimizationFix.md` claims "65% API cost reduction." This assumes probes that pass the local fast-pass filter don't need teacher evaluation. In a safety-critical financial compliance context, is it acceptable to skip teacher evaluation for "approved" outputs? What regulatory frameworks would prohibit this? a142. For anything feeding customer-

facing or compliance decisions, skipping review on a heuristic's say-so is hard to defend. I'm not a lawyer, but frameworks to check include model risk management guidance such as SR 11-7 (validation and effective challenge), supervisory and communications-review rules for regulated firms (FINRA 3110 and 2210 in the US, comparable FCA and MiFID II obligations), and the EU AI Act's risk-management and human-oversight requirements for high-risk uses. Which ones apply depends on the use case. The defensible design is to use the pre-filter for triage and cost control in internal training and eval loops, audit a random sample of locally approved items with a measured miss rate, and keep a decision log.

q143. A probe with `category = "hallucination"` generates high local fluency (r=0.85) because hallucinated text is often grammatically fluent. The fast-pass filter approves it. Explain why high fluency correlation on the local scorer is specifically dangerous for the hallucination detection use case. a143. A high fluency correlation makes the filter look trustworthy, which tempts people to approve on fluency. But fluent text is exactly what LLM hallucinations look like: fluency and truthfulness are largely independent, and confident confabulation often reads better than correct text. So a fluency-based fast-pass approves the worst failure while the metrics look healthy. Approval must require a conjunction: fluency OK and grounding OK and numeric/entity consistency, never fluency alone.

q144. The README badges show `Tests: 18/18 Passed` hardcoded as a static badge. In a real CI/CD pipeline, how would you make this badge dynamic and automatically updated on every push? a144. Add a GitHub Actions workflow that runs `pytest --junitxml` (or `pytest-json-report`) on each push. For pass/fail alone, use the built-in workflow status badge (`.../actions/workflows/ci.yml/badge.svg`). For counts, have the workflow compute passed/total, write a small JSON file to a gist or a branch, and point a shields.io endpoint badge (`img.shields.io/endpoint?url=...`) at it. Coverage can come from Codecov.

q145. The project's Streamlit tab for "Statistical Figures" just shows PNGs with `st.image()`. A quantitative analyst says "I can't drill down into the data from an image." What Plotly/Altair-based implementation would you build instead? a145. Build the scatter and heatmap in Plotly (`st.plotly_chart`) with a `hovertemplate` carrying probe ID, domain, category and the three scores, and use the selection events (`on_select`) so clicking a point shows that probe's text and scores below. Add sliders for the threshold τ so the tradeoff curve updates live, sidebar filters for domain and category, and a CSV export of the filtered rows. Altair with linked brush selections and a table works too.

q146. The `data/human_annotations.json` has a `target_disagreement_axis` field. Why is this field inappropriate to include in a real annotation dataset sent to human raters? What contamination does it cause? a146. It leaks the experimenter's hypothesis: it tells raters where disagreement is "expected", which anchors them and creates demand effects, contaminating independence and distorting agreement. If it is also visible to a judge or a model, it is direct label leakage. Real annotation must be blind to hypotheses, categories

and other raters' scores. Keep generator metadata in a separate hidden file and export only prompt, context and output to Label Studio.

q147. The project stores all probe outputs as a JSON list rather than JSONL (newline-delimited JSON). At what file size does this become a memory problem when loading into Python? What is the practical threshold for switching to JSONL or Parquet? a147. `json.load` builds the full object tree, which typically takes several times the file size in RAM, so a 100 MB file can use around 0.5–1 GB. It becomes a problem at roughly a few hundred MB, and earlier on small machines. Practical rule: switch to JSONL at about 50–100 MB or around 100K records, or whenever you need streaming and appends. Move to Parquet at around a million rows or when you need columnar filtering. At 200 records (under 1 MB) it doesn't matter.

q148. The `run_pipeline.py` orchestrator does not implement parallelism — all 6 steps run sequentially. Identify the two steps that could be parallelized safely, and estimate the speedup achievable on a 4-core machine. a148. After step 1, step 2 (local evaluator) and step 3 (teacher judge) are independent, and within step 3 the Rubric A and Rubric B runs are independent of each other. The five figures in step 6 are also independent. In mock mode the whole run is about 3 seconds, mostly imports and plotting, so by Amdahl's law with about 60% parallelisable work on 4 cores I'd expect roughly 1.5–2×. In live mode the teacher calls dominate and are I/O-bound, so use async concurrency, not cores. Running A and B in parallel alone is about 2×, and per-probe concurrency up to the rate limit gives much more.

q149. The project uses `matplotlib` and `seaborn` for all figures. Both libraries generate static rasterized images. For a financial institution's internal BI dashboard, what argument would you make for SVG vector output rather than PNG? What additional code change is needed? a149. SVG stays sharp at any zoom or print size, is small for line and scatter plots, keeps text selectable and searchable, and embeds cleanly in web dashboards and PDF reports. PNG at 300 DPI is large and blurs when scaled. Code change: `fig.savefig("fig.svg", format="svg")`, with `plt.rcParams["svg.fonttype"] = "none"` to keep text as text. For dense scatter plots with thousands of points, set `rasterized=True` on the scatter artist so the file doesn't balloon.

q150. The project demonstrates that rubric engineering (prompt + exemplar design) improved κ by +0.153 without changing the underlying 70B model. A product manager says: "This means we should always optimize prompts first before buying better models." What nuanced, architecture-aware response would you give to prevent overgeneralizing this finding? a150. "Prompt first" is a sensible default ordering because it is cheap and reversible, but this finding is too narrow to carry "always". It comes from one model, one small synthetic task and mock-mode data where the effect is partly injected, and the +0.153 itself needs reconciling (Q120). The per-axis results show gains on fluency and coherence but not on instruction adherence or hallucination, and those depend on the model's real capability: verifying facts. About three quarters of disagreements in the taxonomy are not ambiguity at all. So I'd set a stopping rule: optimise prompts until κ against human gold

plateaus, then if the remaining errors are knowledge or reasoning limited, invest in retrieval or NLI tools, ensembles, fine-tuning or distillation, or a stronger model. Also remember CoT adds tokens and latency, and prompt gains can break on a model upgrade, so keep prompts under regression tests.

Source files used: README.md, OptimizationFix.md, phases.md, run_pipeline.py, app.py from github.com/ankitghoshthecreator/JudgeLLM, plus the uploaded interviewJudge.md.